

SPARK on Bare Metal

Proving a Real I2C Driver

STM32F746 · BMP390 · Renode, no hardware required

Olivier Henley · AdaCore · AEiC 2026

Goal

Take a working bare-metal I2C driver and prove it has no runtime errors, then watch it run against a simulated sensor.

No new language. No proof scripts. The driver we ship is the driver we prove.

What "no runtime errors" means

Each one is a thing that crashes or corrupts at runtime in C. SPARK proves it cannot happen.

What we rule out	A line that would trip it
integer overflow	<code>X : Natural := Last + 1; -- Last = Natural'Last</code>
out-of-range conversion	<code>B : UInt8 := UInt8 (N); -- N = 300</code>
out-of-bounds access	<code>Buf (I) := 0; -- I past Buf'Last</code>
uninitialized read	<code>return X; -- X was never assigned</code>
loop that never ends	<code>while not Flag loop ... -- Flag may never set</code>
broken precondition	<code>Begin_Write (.., Length $\Rightarrow$ 0); -- Pre: 1 .. 255</code>

Ada in one slide

- Range-checked types. `mod 2**7` , `range 0 .. 1023` , not raw `int` .
- Packages and generics for modularity.
- Contracts in the language: `Pre` and `Post` (what must hold before and after a call), `Global` (what outside state a subprogram may read or write), type predicates.
- Built for high-integrity systems.

SPARK is a provable subset of Ada. Same compiler, same Alire, same code.

SPARK in one slide

What it does

- Proves absence of runtime errors
- Proves contracts you write
- Runs SMT solvers (z3, cvc5, alt-ergo)

What it is not

- A separate language. It is Ada.
- Interactive theorem proving.
- A replacement for testing.

You write code and contracts. GNATprove generates the verification conditions and discharges them.

SMT solver = Satisfiability Modulo Theories. An automated logic engine that decides formulas over arithmetic, arrays, and bit vectors. You hand it a claim, it searches for a counterexample.

Discharge a check = prove it. GNATprove turns each runtime check into a formula meaning "this cannot fail" and asks the solver. No counterexample found, the check is discharged.

The one fact that makes this practical

`SPARK_Mode` is set per unit.

- A spec can be `On` while its body is `Off`.
- Spec `On`, body `Off` reads as: trust this contract, do not prove this body.
- A *register poke* is a direct read or write of a hardware register, like writing `CR2` to start a transfer. SPARK cannot reason about those (volatile memory), and naming the register types drags the whole device binding into the proof.
- So the pokes go in a body marked `Off`. Everything that calls them still gets proven.

We will call this the wall. It is our shorthand. The formal terms are the **trust boundary** and a body marked `SPARK_Mode => Off`.

Volatility

The thing very few people warn you about when you point a prover at a peripheral

A register is not memory

First, the point of view. *Us* is our program, the code the CPU runs in sequence. *The world* is everything else that can touch the register: the peripheral itself, the sensor, interrupts, another bus master.

- The world can change a register between two of our reads, with no write in our code. Read STATUS, it says busy; read again, it says idle.
- Reading a register can have a side effect. Reading the data register pops a byte off the receive FIFO. The act of reading changed the hardware.
- A write we never read back still matters. Writing a command register *is* the action. The compiler must not discard it as unused.

Ada marks all of this with one aspect: `Volatile` .

SPARK refuses to treat it as a single switch.

SPARK splits Volatile into four axes

Axis	Meaning	What the prover must not do
<code>Async_Writers</code>	the world writes it	cache a read, or fold <code>X = X</code> to true
<code>Async_Readers</code>	the world reads it	delete a write whose value you never read
<code>Effective_Reads</code>	reading mutates it	treat a read as side-effect free
<code>Effective_Writes</code>	each write is distinct	coalesce two writes into one

`fold` = compute at compile time and reuse. `coalesce` = merge two operations into one. Both are normal optimizations. On a register, both are wrong.

A plain `Volatile` object has all four set true. Real registers need only some. Naming which ones is the design work.

The four axes, concretely

- `Async_Writers` STATUS. You read it, see busy. The hardware clears it. Your next read sees idle. The prover cannot assume the two reads match, or hoist the read out of a loop.
- `Async_Readers` CR2. You write a START command and never read CR2 back. The hardware still acted on it. The write is not dead code.
- `Effective_Reads` RXDR. Read it once, you consume one byte from the FIFO. Read it twice, you consumed two. The read is a state change.
- `Effective_Writes` TXDR. Write a byte, you send a byte. Write the same byte twice, you send two. The writes cannot be merged.

The four axes are not independent

A coupling means: if the left axis is set, the right one must also be set. Not that every register has both. It is an implication, and it rules out nonsense.

- `Effective_Reads` $\Rightarrow$ `Async_Writers` . RXDR: if reading pops a byte, then the register's contents change without a write of yours. A read that mutates is, by definition, a value the world is altering. You cannot claim one without the other.
- `Effective_Writes` $\Rightarrow$ `Async_Readers` . TXDR: keeping each write distinct only matters if something outside reads every write. Distinct writes with no reader would be pointless.

Other combinations are fine and common. A status flag is `Async_Writers` alone. A timing register is `Async_Readers` plus `Effective_Writes` . The couplings only forbid the contradictory cases.

Consequence one: volatile functions

A status flag has `Async_Writers`, so reading it is a **volatile function**: its result can differ every call, because the world changes it.

That is exactly why SPARK restricts where you may call it. A volatile function may appear only in a **non-interfering context**: alone, as the whole right side of one assignment. Not the conflict it looks like, the consequence.

```
1  -- rejected: two volatile reads in one expression, which value won?
2  exit when Port.Nacked or else Port.Berr;
3
4  -- rejected: a volatile read buried in a condition
5  if Port.Busy then Recover; end if;
6
7  -- correct: take one read, name it, reason about the fixed snapshot
8  declare
9      Bsy : constant Boolean := Port.Busy;
10 begin
11     if Bsy then Recover; end if;
12 end;
```

The rule forces you to pin a single value before you reason about it. Two flags, two locals.

Consequence two: a read that mutates is a procedure

The RX data register has `Effective_Reads` . Reading it pops the FIFO.

In SPARK a function must be free of side effects, because functions are called inside expressions where evaluation order is not pinned down. If reading mutated state, the meaning of an expression would depend on when each read happened. SPARK forbids that outright.

```
1  -- illegal: a function that changes state
2  function Get return Storage_Element;
3
4  -- legal: the effect is explicit and the call is a sequenced statement
5  procedure Get (B : out Storage_Element)
6    with Global => (In_Out => Rx_State);
```

A procedure is called as a statement, in order, with its effect declared in `Global` . That is the only shape that can honestly express a read that changes the world.

Global says what state a subprogram touches

Every operation on the wall declares the abstract state it reads or writes:

```
1  procedure Set_PE (On : Boolean)
2    with Global ⇒ (Output ⇒ Control_State);
3
4  function Busy return Boolean
5    with Global ⇒ (Input ⇒ Status_State),
6         Volatile_Function;
7
8  procedure Get (B : out Storage_Element)
9    with Global ⇒ (In_Out ⇒ Rx_State);
```

Control is written. Status is read. RX is read-and-mutated.

Why it has to agree, under the hood. `Global` is a flow contract: it declares which state each subprogram may read or write. There are no special artifacts. GNATprove does static flow analysis on the body, builds the actual set of reads and writes, and checks it matches what the spec declared. Touch `Status_State` without declaring it and analysis fails. The payoff is modular: a caller reasons about a subprogram from its `Global` alone, never its body, and the body is checked against that contract separately.

The running example

The BMP390

A Bosch barometric pressure and temperature sensor. We use the temperature side.

- A small I2C chip, 7-bit address `0x77` on our board.
- It exposes registers: a chip ID, a calibration block, and a 24-bit raw temperature reading.
- You read its registers over I2C exactly like the protocol on the next slides.
- Cheap, common, and well documented. A realistic target, not a toy.

Our firmware reads its raw temperature, applies the datasheet compensation, and prints degrees. The driver under proof is what does the reading.

The stack

```
1  ghal_examples      app: i2c_bmp (BMP390 over I2C, console over USART)
2    L stm32f746g_disco board instantiation (Instances), Clock_Tree
3      | stm32f746      MCU drivers: I2C, USART, GPIO, SPI
4      |   L STM32F746xx register definitions (svd2ada)
5      |   i2c_generic   generic I2C protocol layer
6      |   usart_generic generic USART protocol layer
7      |   gpio_generic  generic GPIO layer (pin config and AF mux)
8      |   i2c_types     protocol vocabulary: address, speed, ack
9      |   machine_types portable scalars: Bit, UInt2 .. UInt64
10     | light_tasking_stm32f746 runtime
```

Our example drives two peripherals: I2C1 for the sensor, USART1 for the console. Both configure their pins through the GPIO driver first. The layering is the proof boundary: generic protocol on top, MCU specifics in the middle, registers at the bottom.

The driver pattern

Why the code is shaped the way it is, before we prove any of it

Layers, on purpose

Most MCU drivers are one flat module: protocol logic and register pokes tangled together. Ours separates concerns into layers, each with one job.

- **Portable types.** Two crates, two purposes. `machine_types` holds MCU-agnostic scalars (`Bit` , `UInt2` .. `UInt64`). `i2c_types` holds the protocol vocabulary (address, bus speed, ack state). Neither knows about a chip.
- **Control plane.** Configuration and sequencing: enable the peripheral, program timing, issue START and STOP, recover the bus. The policy.
- **Data plane.** The byte movement: `Send` , `Recv` , `Put` , `Get` . The hot path, kept small.
- **Metal.** The register layout and the pokes. The only part that knows it is an STM32F746.

The boundaries between these layers are where SPARK later draws the wall.

Why `machine_types` is its own crate

It would be tempting to put the scalar types inside the HAL. That is a dependency inversion waiting to happen.

- `machine_types` is pure portable definitions. No HAL, no MCU, no dependency. Anything is allowed to depend on it.
- Tie it to the HAL crate instead, and anyone who needs a `UInt32` has to pull the whole HAL in. Wrong direction.
- The clearest case: the `runtime` itself. Its `RCC`, `PWR`, `FLASH` bindings need basic integer types. A dependency-free `machine_types` can serve the runtime. A HAL-bound one cannot, because the HAL sits above the runtime.

Keep the most basic vocabulary at the very bottom, depending on nothing. Then every layer above, including the runtime, can share it.

Control plane and data plane

Why split the two inside the driver:

- **Control** is where the policy and the proof obligations concentrate: the guards, the timing math, the precondition `Length in 1 .. 255`, the recovery logic.
- **Data** is a thin, repeated operation: move one byte. It runs in the inner loop and must stay cheap.
- Separating them keeps the data path tiny and the control path honest. When SPARK later objects, the objections land in the control plane, where the reasoning lives, not scattered through byte pushing.

The orchestration is the portable part

The usual assumption: the protocol is generic, the chip-specific driver just forwards calls. For simple peripherals, fine. For real ones, wrong.

- A peripheral like Ethernet has descriptor rings, state machines, retries, flow control. That orchestration is mostly **identical across MCUs**.
- If you bury it in each chip's driver, you rewrite the hard part every port.
- Put it in a generic layer instead. The orchestration is written once, proven once, and reused. The chip layer only supplies primitives.



Nail the orchestration once and for all. The MCU layer is the thin part, not the generic layer.

The generic layer sees only primitives

`i2c_generic` orchestrates a transfer. It never touches a register. It calls back into formals the instantiator provides.

```
1  generic
2    type Device is limited private;
3
4    with procedure Driver_Begin_Write
5      (Dev : in out Device; Target : I2C_Address;
6       Length : Natural; Stop : Boolean);
7
8    with procedure Driver_Send (Dev : in out Device; B : Storage_Element);
9    with procedure Driver_Recv (Dev : in out Device;
10                               B : out Storage_Element; Ack : Boolean);
11  package I2C_Generic is
12    procedure Write_Read ( ... );
13  end I2C_Generic;
```

The orchestration of `Write_Read` is portable. The primitives are not. The seam between them is explicit.

Generics, not dispatch

We could have made the driver an abstract tagged type with dispatching operations. We chose generics. The difference is where specialization happens.

- A generic is specialized at compile time. `new STM32F746_I2C ( ... )` stamps out one concrete copy with the formals bound in. Like a C++ template or a Rust generic.
- Every call inside it is a direct call. No virtual table, no indirect jump. The compiler can inline straight through.
- A tagged type dispatches at runtime through a vtable. One shared body, but the call target is decided at runtime.

Generics erase the dispatch. There is no runtime cost to the abstraction, and the timing is deterministic, which matters on bare metal.

The tradeoff, honestly

Generics are not free. You pay in code size.

- Each instantiation generates its own specialized copy of the code. Two I2C buses, two copies. This is monomorphization, the same bloat C++ templates have.
- Dispatch would give you one shared body and smaller code, at the price of indirect calls and runtime cost.

For SPARK the generic choice pays twice over:

- The call graph is static and exact. The prover knows precisely which body runs.
- No class-wide dispatch means no LSP obligation. LSP, the Liskov Substitution Principle, says any override must honor the parent's contract. With dispatch the prover must verify that for every override. Generics have none of it. Less aliasing to reason about, too.

Heavier binary, lighter proof, zero runtime overhead. For an embedded driver that is the right side of the trade.

Generics next to C++ and Java

The same idea exists in three languages, with different mechanics.

	Ada generic	C++ template	Java generic
When	compile time	compile time	compile time only
Mechanism	monomorphization	monomorphization	type erasure
At runtime	one copy per instance	one copy per instance	one shared copy
Cost	code size	code size	boxing and casts
The contract	explicit formal part	implicit, or C++20 concepts	bounded type params, then erased
Instantiation	explicit <code>new</code>	implicit on use	implicit, erased

- Ada and C++ both specialize per instantiation: no runtime cost, bigger binary. Java erases generics to a single `Object`-based body: small binary, runtime casts and boxing, no real primitive specialization.

Ergonomics: the Driver_ prefix

The formals are named `Driver_Begin_Write`, `Driver_Send`, `Driver_Recv`.

- Inside the generic, a call to `Driver_Send` reads as "this is the hardware primitive I was handed", not a local procedure.
- It keeps the orchestration code honest about where the metal boundary is.
- At instantiation, the mapping is one obvious block: every `Driver_*` formal binds to the MCU driver's matching operation.

Small thing. It makes the call graph legible to a reader and to the prover.

Two interfaces, two audiences

Layer	Operations	Audience
MCU driver (primitives)	Init , Enable , Disable , Begin_Write , Begin_Read , Send , Recv	the generic orchestrator
Public client interface	Open , Close , Reset , Write_Read	application code

The application says `Open` then `Write_Read`. It never sees `Begin_Write` or a register. The client point of view stays clean: a bus you open, talk to, and close.

Why Get_Clock, RCC_Enable, RCC_Reset are formals

The I2C driver is generic over more than the peripheral address:

```
1  generic
2      Base      : System.Address;
3      with function Get_Clock return Natural;
4      with procedure RCC_Enable;
5      with procedure RCC_Reset;
6  package STM32F746_I2C is ...
```

- The driver needs its kernel clock to compute `TIMINGR` , and needs to be enabled and reset through the RCC.
- But the clock tree and the RCC are **chip-global resources**. A peripheral driver has no business owning them.
- So it asks for them. "Tell me my clock. Enable me. Reset me." The board decides how.

Why Clock_Tree lives at the board level

```
1  Clock_Tree (one PLL config → every derived clock)
2      | Get_I2C1_Clock → 54_000_000
3      | Enable_I2C1   → sets the RCC bit
4      | Reset_I2C1    → pulses the RCC reset
```

- One source of truth. The PLL config is computed once, and every peripheral clock derives from it. The 54 MHz the I2C driver proves against is the same number the runtime is configured with.
- Portability. The same I2C driver runs on any board or clock setup, because it hardcodes nothing. The board supplies the clock.
- Proof. The harness can pass a neutral `Get_Clock` returning 54 MHz without dragging the real RCC into the analysis.

The peripheral driver stays a leaf. The chip-global concern stays central.

I2C, just enough

- Two wires, SDA and SCL. One controller, many peripherals.
- Each peripheral has a 7-bit address. Our BMP390 is `0x77`.
- You talk to registers: write an address, then read or write data.

Reading the chip ID register `0x00` :

```
1  START · 0x77+W · 0x00 · repeated-START · 0x77+R · [0x60] · STOP
```

- `START` we take the bus.
- `0x77+W` address the BMP390, write direction. We are about to tell it something.
- `0x00` the register we want to read from. The chip now has its pointer set to `CHIP_ID`.
- `repeated-START` start again without releasing the bus, so no other master can jump in between setting the pointer and reading.
- `0x77+R` same chip, now read direction.
- `[0x60]` the chip drives the byte back. `0x60` is the BMP390's known chip ID.
- `STOP` release the bus.

Write the register address, then read the value back. Our `Write_Read` is exactly this shape.

What is Renode

A full-system emulator from Antmicro. It runs your **unmodified firmware binary** against a model of the whole board.

- Models the CPU and the peripherals: I2C, USART, GPIO, timers, the lot.
- Open source, MIT licensed. Written in C# on .NET.
- You script a machine, load your ELF, and drive peripherals from a monitor.
- No board, no probe, no soldering. The same firmware that runs here runs on metal.

For a tutorial it means everyone in the room runs the exact same hardware, deterministically.

Installing Renode on Linux

Releases: `github.com/renode/renode/releases/latest`

The portable archive embeds its own .NET runtime, nothing else to install:

```
1  mkdir renode_portable
2  wget https://builds.renode.io/renode-latest.linux-portable.tar.gz
3  tar xf renode-latest.linux-portable.tar.gz \
4      -C renode_portable --strip-components=1
5  cd renode_portable
6  export PATH="`pwd`:.$PATH"
7  renode
```

Or grab the `.deb` / `.rpm` / `.pkg.tar.xz` from the same releases page. Those need dotnet ≥ 6.0 on the host.

How it runs: Renode, not hardware

We emulate the whole board, CPU and peripherals. Inside `i2c_bmp.resc` :

```
1  include @bmp390.cs                # our own BMP390 model
2  machine LoadPlatformDescription @stm32f746-bmp390.repl
3  showAnalyzer sysbus.usart1
4  macro reset "sysbus LoadELF @i2c_bmp.elf"
5  runMacro $reset
```

Launch it, then drive the sensor from the monitor:

```
1  renode
2  (monitor) i @i2c_bmp.resc
3  (monitor) start
4  (monitor) sysbus.i2c1.bmp390 RawTemperature 8200000
```

Renode is not the proof tool. GNATprove gives static proof. Renode gives executable feedback. We use both.

We wrote the sensor too

The BMP390 is a 150-line C# model: register pointer, auto-increment, soft reset, a pokeable raw temperature.

```
1 public void Write(byte[] data) {  
2     foreach (var b in data) {  
3         if (collectingAddress) { pointer = b; collectingAddress = false; }  
4         else { WriteRegister(pointer, b); pointer++; } // auto-increment  
5     }  
6 }
```

Same doctrine, applied to the peripheral side. The model is small and exposed.

The three files that make the emulation

```
1  bmp390.cs           our BMP390 peripheral model (C#)
2  stm32f746-bmp390.repl platform: the disco board + the sensor on I2C1
3  i2c_bmp.resc        the run script: load model, load ELF, start
```

- The `.repl` (platform description) places our `bmp390` on the `i2c1` bus of the STM32F746 disco machine Renode already knows.
- The `.cs` is the sensor behavior. Renode compiles it on load.
- The `.resc` is what you actually run: it wires the model in, loads the firmware ELF, and starts the CPU.

All three are in the tutorial materials repo:

```
1  github.com/GNAT-Academic-Program/aeic_2026_start_spark
```

```
1  renode
2  (monitor) i @i2c_bmp.resc      # include and run the script
3  (monitor) start
```

Get the code

Everything is in the GNAT Academic Program org. Alire resolves the dependency graph from the top crate down.

```
1  # the apps (top of the stack)
2  git clone https://github.com/GNAT-Academic-Program/ghal_examples.git
3
4  # pulled in transitively, or clone to follow along:
5  #   stm32f746g_disco  board instantiation
6  #   stm32f746_ghal    MCU drivers (this repo holds the tut/* branches)
7  #   i2c_generic, machine_types, light-tasking-stm32f746
8
9  cd ghal_examples
10 alr build          # resolves and fetches the rest
```

The four tutorial branches live in `stm32f746_ghal`: `tut/01-leak` through `tut/04-volatile`.

Tutorial materials, the Renode `.resc` / `.repl` / `.cs` and the starting point:

`github.com/GNAT-Academic-Program/aeic_2026_start_spark`

The conversion, in four branches

`git checkout` each one and run the prover live

Branch 01: the leak

```
1  git checkout tut/01-leak
2  alr gnatprove --level=2 --prover=z3,cvc5 -u src/i2c/i2c_leak_demo.ads
```

We instantiate the original flat driver inside a SPARK unit.

```
1  package I2C_Leak_Demo with SPARK_Mode ⇒ On is
2    package Inst is new STM32F746_I2C
3      (Periph      ⇒ STM32F746.I2C.I2C1_Periph'Access,
4       Get_Clock   ⇒ Clock_Tree.Get_I2C1_Clock,
5       RCC_Enable  ⇒ Clock_Tree.Enable_I2C1,
6       RCC_Reset   ⇒ Clock_Tree.Reset_I2C1);
7  end I2C_Leak_Demo;
```

GNATprove stops at the formal:

```
1  access to "I2C_Peripheral" is not allowed in SPARK
2    (incompatible designated type and access type with respect to volatility
3     due to Async_Readers, Async_Writers, Effective_Reads, Effective_Writes)
```

Slide one already names the four axes. This is the whole talk in an error message.

Branch 01: the leak is in the contract

The flat driver carries the peripheral pointer in its spec:

```
1  generic
2    Periph : not null access stm32f746.I2C.I2C_Peripheral;
3    ...
4  package STM32F746_I2C with SPARK_Mode ⇒ On is
5    ...
6    P : access stm32f746.I2C.I2C_Peripheral := Periph;  -- in a private field
```

- The register pokes are fine. They live in the body, behind the wall.
- SPARK never reached them. It rejected the pointer in the interface.
- A spec is the contract every caller sees. The hardware coupling leaked out of the body, through the spec, into everyone's proof.

Branch 01: why the pointer is rejected

It is not a guess. SPARK reads a property off the type.

- The SVD record `I2C_Peripheral` is declared `Volatile` , which carries all four axes.
- A pointer's volatility must match what it designates. The pointer reads as ordinary, the target reads as volatile. Mismatch.
- Deeper reason: SPARK's pointer ownership model is built for normal RAM. Volatile MMIO changes from outside and reads have effects, so SPARK excludes owning pointers to it entirely.

You cannot own a pointer to memory the hardware rewrites underneath you.

Branch 02: move the pointer out of the contract

```
1  git checkout tut/02-split
2  alr gnatprove --level=2 --prover=z3,cvc5 -u src/i2c/stm32f746_i2c.ads
```

Replace the access formal with a plain address. The SVD type disappears from the spec.

```
1  generic
2    Base      : System.Address;          -- a number, SPARK-clean
3    with function Get_Clock return Natural;
4    with procedure RCC_Enable;
5    with procedure RCC_Reset;
6  package STM32F746_I2C with SPARK_Mode ⇒ On is
7    type Device is limited private;
8    ...
```

`System.Address` has no designated type and no volatility.
There is nothing for the matching rule to reject.

Branch 02: the wall, made concrete

The volatile type now lives in exactly one place: the port body, marked `Off`.

```
1  -- spec: SPARK_Mode ⇒ On, only clean types cross this line
2  package STM32F746_I2C_Port with SPARK_Mode ⇒ On is
3      ...
4  end STM32F746_I2C_Port;
5
6  -- body: SPARK_Mode ⇒ Off, the only place that names the SVD type
7  package body STM32F746_I2C_Port with SPARK_Mode ⇒ Off is
8      Regs : I2C_Peripheral
9          with Import, Volatile, Address ⇒ Base;    -- overlay at the base address
10     ...
11 end STM32F746_I2C_Port;
```

Branch 02: the axes become a committed table

An `Abstract_State` is a name for state hidden in the body. `with External` marks that state as talking to the world, the hardware, not ordinary memory. The properties listed are which volatility axes apply to it.

```
1  package STM32F746_I2C_Port with
2    SPARK_Mode ⇒ On,
3    Abstract_State ⇒
4      ((Control_State with External ⇒ (Async_Readers, Effective_Writes)),
5       (Status_State with External ⇒ Async_Writers),
6       (Rx_State with External ⇒ (Async_Writers, Effective_Reads)))
7  is
```

- `Control_State` : we write, hardware reads. `Async_Readers` , `Effective_Writes` .
- `Status_State` : hardware writes, we read. `Async_Writers` .
- `Rx_State` : hardware writes, reading pops the FIFO. `Async_Writers` , `Effective_Reads` .

Three named external states, each with its axes declared. Not a comment. The compiler holds the body to it.

The registers we actually touch

Reg	Full name	Role in the I2C protocol
CR1	Control Register 1	enable the peripheral (PE)
CR2	Control Register 2	address, direction, count, START, STOP
TIMINGR	Timing Register	SCL timing from the kernel clock
ICR	Interrupt Clear Register	write-1 to clear status flags
ISR	Interrupt and Status Register	the status flags we poll
TXDR	Transmit Data Register	the byte to send
RXDR	Receive Data Register	the received byte, reading pops it

The registers, mapped to the four axes

Registers	A_Writers	A_Readers	E_Reads	E_Writes	State
CR1 CR2 TIMINGR ICR TXDR		x		x	Control
ISR	x				Status
RXDR	x		x		Rx

The write registers share the output axes. `ISR` is the pure input. `RXDR` is the one that mutates on read. The three abstract states fall straight out of this table.

Branch 02: but the prover stays quiet

Run gnatprove on the leaf spec and it does nothing useful:

- 1 generic compilation unit is not analyzed
- 2 only instantiations of the generic will be analyzed

A generic has no bound formals yet. There is no `Base` , no clock, nothing concrete to check.

Proof needs an instantiation.

Branch 03: give it an instantiation

```
1  git checkout tut/03-harness
2  alr gnatprove --level=2 --prover=z3,cvc5 -u src/i2c/i2c_proof_instance.ads
```

A neutral harness binds the formals with stubs. Nothing real, just enough to analyze.

```
1  with STM32F746_I2C;
2  with System; use System;
3
4  package I2C_Proof_Instance with SPARK_Mode ⇒ On is
5      function No_Clock return Natural is (54_000_000);
6      procedure No_Op is null;
7
8      package Inst is new STM32F746_I2C
9          (Base      ⇒ System.Null_Address,
10           Get_Clock ⇒ No_Clock,
11           RCC_Enable ⇒ No_Op,
12           RCC_Reset  ⇒ No_Op);
13  end I2C_Proof_Instance;
```

`Null_Address` , not `'Address` . The attribute is illegal in SPARK. A null address is a clean constant.

Branch 03: now it pushes back

The prover analyzes the body and rejects code that compiles and runs:

```
1  call to a volatile function in interfering context
2    is not allowed in SPARK
```

at every line like:

```
1  if Port.Busy then ...
2  exit when Port.Nacked or else Port.Berr or else Port.Arlo or else Port.Ovr;
```

This is the volatile-function rule from earlier, hitting real code. `Async_Writers` means a flag read inside an expression is not allowed.

Branch 04: snapshot every read

```
1  git checkout tut/04-volatile
2  alr gnatprove --level=2 --prover=z3,cvc5 -u src/i2c/i2c_proof_instance.ads
```

Read each flag into a local first, then reason about the local.

```
1  -- before
2  if Port.Busy then Port.Recover_Ctrl; end if;
3
4  -- after
5  Bsy := Port.Busy;
6  if Bsy then Port.Recover_Ctrl; end if;
```

The polling loop reads all flags into locals, one declaration each:

```
1  declare
2    F_Nack : constant Boolean := Port.Nacked;
3    F_Berr : constant Boolean := Port.Berr;
4    F_Arlo : constant Boolean := Port.Arlo;
5    F_Ovr  : constant Boolean := Port.Ovr;
6  begin
7    exit when F_Nack or else F_Berr or else F_Arlo or else F_Ovr;
8  end;
```

Branch 04: green

1	SPARK Analysis results	Total	Flow	Provers	
2					
3	Initialization	9	9		
4	Run-time Checks	10		10 (Z3)	
5	Termination	3	3		
6					
7	Total	22	12	10	0 unproved

And in the detailed report, every port operation looks like this:

```
1  Inst.Port.Busy  skipped; body is SPARK_Mode ⇒ Off
2  Inst.Port.Get   skipped; body is SPARK_Mode ⇒ Off
3  Inst.Port.Start skipped; body is SPARK_Mode ⇒ Off
```

- The 10 runtime checks: the `UInt10` address widening, the `UInt8` length, the discriminant on the timing result.
- The 3 terminations: the bounded `for` loops.
- Every `Port.*` operation is skipped because its body is `Off`. That is the wall, in the output: trusted, not analyzed.

Two more things the proof forced

A total timing function, no raise

```
1  type Timing_Result (Valid : Boolean := False)
2  is record
3    case Valid is
4      when True  => T : Timing;
5      when False => null;
6    end case;
7  end record;
```

An unknown clock returns `Valid => False`.

`Init` faults at the boundary, not inside.

A bounded loop, not a while

```
1  for Count in 1 .. Timeout_Loops loop
2    ...
3    exit when Flag;
4  end loop;
```

A flag that never sets becomes a defined timeout, not a hang. Termination is free.

The timing table is keyed on PCLK1 = 54 MHz. The board config and the proof are pinned to each other.

Error reporting also lives behind the wall

The runtime sets `No_Exception_Propagation`. A raise does not return to the caller.

```
1  procedure Fault (Why : String) with No_Return;  
2  
3  procedure Fault (Why : String) with SPARK_Mode ⇒ Off is  
4  begin  
5      raise I2C_Types.Bus_Fault with Why;  -- routes to Last Chance Handler, halts  
6  end Fault;
```

`No_Return` tells SPARK every guard-failure path is unreachable, so it proves them trivially. The one runtime-coupled act, the raise, sits in an `Off` body. Same wall, applied to faults.

A subtle lesson from the warnings

Branch 04 is green, but it warns:

```
1 statement has no effect ... Fault ( ... )  
2 subprogram "Check_Errors" has no effect
```

- These checks are fully proven, not pending. The harness binds a valid clock, so `Timing_For` always hits a row, so the fault path is provably unreachable.
- That is the dead code SPARK is reporting. Real proof, of a degenerate case.
- A real `Get_Clock` can return an unsupported value. Now the fault path is reachable, and SPARK generates a check the harness made vacuous.

A generic has no single proof. It has one per instantiation. The harness proves the board-independent core for real: arithmetic, ranges, loops, the volatile discipline. A real instance adds the input-dependent checks the stub makes trivial.

Then the firmware just builds

The board instantiation changes one line:

```
1 package I2C_1 is new STM32F746_I2C
2   (Base      ⇒ STM32F746.I2C.I2C1_Periph'Address,    -- Base, not Periph
3     Get_Clock ⇒ Clock_Tree.Get_I2C1_Clock,
4     RCC_Enable ⇒ Clock_Tree.Enable_I2C1,
5     RCC_Reset  ⇒ Clock_Tree.Reset_I2C1);
```

The register pokes are byte-identical to the flat driver. Same bytes on the wire. We moved the proof boundary, not the behavior. Renode prints temperatures.

```
1 cd ghal_examples
2 alr build                # builds the i2c_bmp firmware
3
4 renode                  # start Renode
5 (monitor) i @i2c_bmp.resc # include and run the script
6 (monitor) start          # run the CPU
7 # then poke the simulated sensor:
8 (monitor) sysbus.i2c1.bmp390 RawTemperature 8200000
```

The proof boundary, stated honestly

1	+-----+	
2	i2c_generic · BMP390 · machine_types	PROVEN
3	STM32F746_I2C leaf body (logic, conversions,	
4	guards, loops, volatile snapshots)	
5	+-----+	
6	STM32F746_I2C_Port spec (contracts, axes)	ASSUMED
7	+-----+	
8	STM32F746_I2C_Port body (overlay, register	TRUSTED
9	pokes, the No_Return raise) · SVD bindings	
10	+-----+	
11	silicon · board wiring · sensor physics	OUTSIDE
12	+-----+	

This is the move. In the flat driver the whole MCU body was trusted. The split pushed the wall down: the driver logic is now **proven**, and only the register-poke port body stays trusted.

Every proof system has a trust base. Coq trusts its kernel. seL4 trusts the compiler. SPARK trusts the hardware layer and runtime. Naming it is the strength, not the gap.

Two tools, two jobs

Tool	Gives us
GNATprove	static proof of contracts and absence of runtime errors
Renode	executable behavior against a simulated board and sensor

GNATprove does not prove the Renode model is faithful.

Renode does not prove absence of runtime errors.

Together: static confidence plus executable feedback.

Why SPARK, and not a proof assistant

The other tools prove a different artifact

	Coq · Lean · Agda	SPARK
You write	proof terms or tactics	code plus contracts
Expressiveness	maximal	bounded, automated
What runs	code extracted to OCaml, Haskell, Scheme	the Ada you wrote
Runtime	garbage collected	none, you control it
Resources	time and space not under your control	deterministic, no GC pauses

A proof assistant proves a model, then extracts code onto a managed runtime. SPARK proves the systems code you compile and ship.

The rewrite gap

The usual high-assurance pipeline has a seam in the middle:

```
1  verify a model          then      reimplement in the real language
2  (Coq, Lean, ... )      (C, C++, Ada, ... )
3      proven              shipped
4      \___ the gap ___/
5      correctness does not cross for free
```

- Some tools verify in a proof-oriented language and then **generate** C from it. That is better, but the C is what ships, and the generator becomes part of what you have to trust.
- Others verify a model that is **refined down** to the real code, which is a large, separate proof effort on its own.
- Either way there is a translation step between the thing proven and the thing deployed. Correctness has to be carried across that step.

SPARK has no seam

The proof obligations are discharged on the **actual compiled Ada**.

- No extraction. No model-to-implementation refinement. The proven source is the shipped source.
- No GC, predictable timing and memory. It is a systems language doing systems work.
- Qualifiable for DO-178C, EN 50128, and similar. The certification story is about the binary you fly, not a model of it.



That is the ergonomic win. The proof rides on the production code. You do not prove one thing and deploy another.

Where this is heading

AdaCore is preparing an AI platform for Ada and SPARK developers.

- The bet is the feedback loop. Ada and SPARK already produce high-quality signal: strong types, explicit specs, contracts, proof results. An assistant has far more to work from here than in untyped, uncontracted code.
- The goal: keep developers on the latest productivity gains, and help them produce better quality code faster.
- Proper announcements are expected this coming autumn.

The same contracts that let GNATprove reason precisely are exactly what an AI tool needs to assist precisely.

Recap

- A real I2C driver, proven with no runtime errors, running in emulation.
- The leak was a pointer to volatile hardware in a SPARK contract. The fix was to pass an address and keep the volatile type behind an `Off` wall.
- SPARK's four volatility axes drove the design: status reads are volatile functions, the RX read is a procedure, every flag gets snapshotted.
- A generic proves only through an instantiation. Proof is instantiation-relative.
- The split pushed the wall down: the driver logic is now proven, and only the register-poke port body stays trusted. Small, explicit, documented.

Resources

Four branches to walk: `tut/01-leak` · `tut/02-split` · `tut/03-harness` · `tut/04-volatile`

`learn.adacore.com` · SPARK User's Guide · Alire (`alr`)

`github.com/GNAT-Academic-Program/stm32f746_ghal`
`github.com/GNAT-Academic-Program/aeic_2026_start_spark`

Thank you

Questions, and let's run it.

Olivier Henley · AdaCore · AEiC 2026